Cryptology

LAB Assignment 3

Name: Shreyas S Sanil

Roll No: 261100690003

Subject: Cryptology

Part A — Python Basics and Debugging

Question 1: Debug the following programs.

Python Code:
```
print("Welcome to Cryptology Lab")
message = "Python Programming"
print(message)
num = int(input("Enter a number: "))
result = num + 10
print(result)
```

Sample Output:
```
Welcome to Cryptology Lab
Python Programming
Enter a number: 123
133
```

Question 2: Write Python statements to display the following exactly:
Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab".

Python Code:
```
print("Welcome to Cryptology Lab")
print("It's interesting to learn Python.")
print('Students, "Welcome to the Cryptology Lab".')
```

Sample Output:
```
Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab".
```

Question 3: Read two numbers from the user and perform addition, subtraction, multiplication, division, integer division, remainder, exponentiation. Display each result with an appropriate message.

Python Code:
```
n1 = float(input("Enter n1: "))
n2 = int(input("Enter n2: "))

print("Addition:", n1 + n2)
print("Subtraction:", n1 - n2)
print("Multiplication:", n1 * n2)
print("Division:", n1 / n2)
print("Integer Division:", n1 // n2)
print("Remainder:", n1 % n2)
print("Exponentiation:", n1 ** n2)
```

Sample Output:
```
Enter n1: 101
Enter n2: 10
Addition: 111.0
Subtraction: 91.0
```

```
Multiplication: 1010.0
Division: 10.1
Integer Division: 10.0
Remainder: 1.0
Exponentiation: 1.1046221254112045e+20
```

Question 4: Read a floating-point number and display its data type, its absolute value, and the value rounded to two decimal places.

PythonCode:

```
num = float(input("Enter a floating-point number: "))
print("Data type:", type(num))
print("Absolute value:", abs(num))
print("Rounded(2 decimal):", round(num, 2))
```

Sample Output:

```
Enter a floating-point number: -122.212343
Data type: <class 'float'>
Absolute value: 122.212343
Rounded(2 decimal): -122.21
```

Part B — Data Types and Collections

Question 5: Create variables containing examples of int, float, complex, str, list, tuple, set, and dictionary. Display value and data type of each variable.

PythonCode:

```
a = 100
b = 2.3
cnum = 2 + 3j
s = "Python"
nlist = [1, 2, 3]
tup = (1, 2, 3)
nset = {1, 2, 3}
dct = {"name": "John", "age": 20}

print(a, type(a)) print(b,
type(b))       print(cnum,
type(cnum))       print(s,
type(s))      print(nlist,
type(nlist))    print(tup,
type(tup))     print(nset,
type(nset))     print(dct,
type(dct))
```

Sample Output:

```
100 <class 'int'>
2.3 <class 'float'>
(2+3j) <class 'complex'>
Python <class 'str'>
[1, 2, 3] <class 'list'>
(1, 2, 3) <class 'tuple'>
```

```
{1, 2, 3} <class 'set'>
{'name': 'John', 'age': 20} <class 'dict'>
```

Question 6: Demonstrate experimentally that strings are immutable, whereas lists are mutable.

PythonCode:
```
message = "Python"
# message[0] = "J" # Raises TypeError: 'str' object does not support item
assignment

numbers = [1, 2, 3]
numbers[0] = 10
print("Updated:", numbers)
```

Sample Output:
```
Updated: [10, 2, 3]
```

Question 7: Create a list containing five prime numbers. Display first three elements, last element, append another prime number, extend with two more, and display total count.

PythonCode:
```
prime = [2, 3, 5, 7, 11]

print("First three:", prime[:3])
print("Last:", prime[-1])

prime.append(13)
print("After append:", prime)

prime.extend([17, 19])
print("After extend:", prime)

print("Total:", len(prime))
```

Sample Output:
```
First three: [2, 3, 5]
Last: 11
After append: [2, 3, 5, 7, 11, 13]
After extend: [2, 3, 5, 7, 11, 13, 17, 19]
Total: 8
```

Part C — String Operations

Question 8: For message = "CRYPTOLOGY USING PYTHON", display length, first 5 characters, last 6 characters, positions 3 to 8, and reverse order.

PythonCode:
```
message = "CRYPTOLOGY USING PYTHON"
print("Length:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
```

```
print("Characters from positions 3 to 8:", message[3:9])
print("Reverse:", message[::-1])
```

Sample Output:

```
Length: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from positions 3 to 8: PTOLOG
Reverse: NOHTYP GNISU YGOLOTPYRC
```

Question 9: Convert to lowercase, convert to uppercase, check whether "PYTHON" exists, and replace "PYTHON" with "PROGRAMMING".

PythonCode:

```
message = "CRYPTOLOGY USING PYTHON" print("Lower:",
message.lower())
print("Upper:", message.upper())
print("PYTHON" in message)
updated = message.replace("PYTHON", "PROGRAMMING")
print("After replacement:", updated)
```

Sample Output:

```
Lower: cryptology using python
Upper: CRYPTOLOGY USING PYTHON
True
After replacement: CRYPTOLOGY USING PROGRAMMING
```

Question 10: Remove the spaces from the string and display the resulting string.

PythonCode:

```
message = "CRYPTOLOGY USING PYTHON"
print(message.replace(" ", ""))
```

Sample Output:

```
CRYPTOLOGYUSINGPYTHON
```

Question 11: Using a loop, display every character and its corresponding Unicode/ASCII value using ord().

PythonCode:

```
message = "CRYPTOLOGY USING PYTHON"
for c in message:
    print(c, "->", ord(c))
```

SampleOutput:

```
C -> 67
R -> 82
Y -> 89
P -> 80
T -> 84
O -> 79
L -> 76
```

```
O -> 79
G -> 71
Y -> 89
-> 32 U
-> 85 S
-> 83 I
-> 73 N
-> 78 G
->   71
-> 32 P
-> 80 Y
-> 89 T
-> 84 H
-> 72 O
-> 79 N
-> 78
```

Question 12: Use chr() to convert values [65, 67, 73, 80, 72, 69, 82] into characters, combine them, and identify the word.

PythonCode:

```
num = [65, 67, 73, 80, 72, 69, 82]
res = ""
for number in num:
    res += chr(number)
print("Result:", res)
```

Sample Output:

```
Result: ACIPHER
```

Part D — Text Processing for Cryptology

Question 13: Plaintext preprocessing: Read a message from user and remove leading/trailing spaces, convert to uppercase, remove spaces between words.

PythonCode:

```
message = input("Enter message: ")
message = message.strip().upper().replace(" ", "")
print("Result:", message)
```

Sample Output:

```
Enter message: meet me tomorrow
Result: MEETMETOMORROW
```

Question 14: Character grouping: Read a string and divide it into blocks of five characters.

PythonCode:

```
message = input("Enter a string: ")
for i in range(0, len(message), 5):
    print(message[i:i+5])
```

Sample Output:
```
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LAB
```

Question 15: Padding: Modify Question 14 so that an incomplete final block is padded with 'X'.

Python Code:
```
message = input("Enter a string: ")
for i in range(0, len(message), 5):
    block = message[i:i+5]
    if len(block) < 5:
        block = block + "X" * (5 - len(block))
    print(block)
```

Sample Output:
```
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LABXX
```

Question 16: Character frequency: Calculate character frequencies for "CRYPTOLOGYISINTERESTING" and identify the most frequently occurring character.

Python Code:
```
text = "CRYPTOLOGYISINTERESTING"
frequency = {}
for c in text:
    if c in frequency:
        frequency[c] += 1
    else:
        frequency[c] = 1

max_freq = max(frequency, key=frequency.get)

print("Most frequent:")
print(max_freq, " : ", frequency[max_freq])
```

Sample Output:
```
Most frequent:
T : 3
```

Question 17: Percentage frequency: Extend Question 16 to calculate percentage frequency of each character rounded to two decimal places.

Python Code:
```
text = "CRYPTOLOGYISINTERESTING"
frequency = {}
total = len(text)

for ch in text:
```

```
    if ch in frequency:
        frequency[ch] += 1
    else:
        frequency[ch] = 1

for   ch,   freq   in   frequency.items():
percent = (freq / total) * 100 print(ch,
" : ", round(percent, 2))
```

SampleOutput:

```
C  :  4.35
R  :  8.7
Y  :  8.7
P  :  4.35
T  :  13.04
O  :  8.7
L  :  4.35
G  :  4.35
I  :  13.04
S  :  8.7
N  :  8.7
E  :  8.7
```

Part E — Mathematical Foundations for Cryptology

Question 18: Modular arithmetic: Evaluate 29mod26, 55mod26, 78mod26, -3mod26, -29mod26 and write a program that calculates modulo 26.

PythonCode:

```
num = int(input("Enter a number: "))
print("Mod: ", num % 26)

nums = [29, 55, 78, -3, -29]
for n in nums:
    print(n, "mod26 = ", n % 26)
```

Sample Output:

```
Enter a number: 22
Mod: 22
29 mod26 =  3 3
55 mod26 =  0
78 mod26 =  23
-3 mod26 =
-29 mod26 =  23
```

Question 19: Alphabet representation (A=0, B=1, ... Z=25): Convert uppercase character to number and number to uppercase character.

PythonCode:

```
# Character to number
char = input("Enter uppercase character: ")
number = ord(char) - ord("A")
print(char, "->", number)
```

```
#Number to character
num=int(input("Enter number: "))
c=chr(num + ord('A'))
print(num, "->", c)
```

Sample Output:

```
Enter uppercase character: D
D -> 3
Enter number: 25
25 -> Z
```

Question 20: Prime numbers: Determine whether a given number is prime and display all primes in a specified range.

PythonCode:

```
n = int(input("Enter a number: "))
if n < 2:
    print("Not prime number")
else:
    prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            prime = False
            break
    if prime:
        print("Prime Number")
    else:
        print("Not a prime number")

start = int(input("Enter num1: "))
end = int(input("Enter num2: "))
for num in range(start, end + 1):
    if num < 2:
    continue
    prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            prime = False
            break
    if prime:
        print(num, end=" ")
print()
```

Sample Output:

```
Enter a number: 17
Prime Number
Enter num1: 1
Enter num2: 20
2 3 5 7 11 13 17 19
```

Question 21: GCD and coprimes: Read two integers from the user. Find their GCD and determine whether they are coprime.

Python Code:

```
num1 = int(input("Enter num1: "))
num2 = int(input("Enter num2: "))
while num2 != 0:
    num1, num2 = num2, num1 % num2

print("GCD:", num1)
if num1 == 1:
    print("Numbers are coprime")
else:
    print("Numbers are not coprime")
```

Sample Output:

```
Enter    num1:    15
Enter num2: 26 GCD:
1
Numbers are coprime
```

Question 22: Write a program to get the multiplicative inverse of an integer b modulo 26.

Python Code:

```
b = int(input("integer b: "))
inverse = -1
for x in range(1, 26):
    if (b * x) % 26 == 1:
        inverse = x
        break

if inverse != -1:
    print("Multiplicative inverse of", b, "= ", inverse)
else:
    print("Multiplicative inverse does not exist.")
```

Sample Output:

```
integerb: 7
Multiplicative inverse of 7 =  15
```